BOOTCAMP AI WEB PROGRAMMING · MODUL BACAAN SESI 9 · MODUL 2 DARI 2

Membaca SQL dari Agen

Cara memeriksa *database function*, *trigger*, dan *policy* sebelum menekan tombol setuju

Dari membedah fungsi `catat_stok_masuk` baris demi baris, empat hal di daftar periksa SQL, cara kerja *trigger*, contoh SQL yang harus ditolak, sampai menyimpan, memeriksa, dan menguji SQL yang sudah disetujui di aplikasi Bengkel Cemerlang Jaya Motor.



Ioda Academy · Oktober 2026

Daftar Isi

Bab 1 · Kenapa SQL dari Agen Perlu Dibaca?

1.1 Satu Database untuk Empat Anggota
1.2 Kesalahan yang Sering Ada di SQL Agen
1.3 Kenapa SQL yang Dijalankan Tidak Bisa Ditarik?
1.4 Urutan Membaca dan Menyetujui SQL

Bab 2 · Cara Membaca *Database Function*

2.1 Apa Itu *Database Function*?
2.2 Bagaimana Aplikasi Memanggil Fungsi?
2.3 Bedah Fungsi `catat_stok_masuk`
2.4 Apa yang Terjadi Saat Fungsi Menolak?
2.5 Siapa yang Boleh Menjalankan Fungsi?

Bab 3 · Empat Hal di Daftar Periksa SQL

3.1 Apa Arti `security definer`?
3.2 Kenapa `search_path` Harus Kosong?
3.3 Kenapa Peran Diperiksa di Awal?
3.4 Pemeriksaan Tim di `ubah_status_wo`
3.5 Aturan Perpindahan Status di Dalam Fungsi
3.6 Kapan Baris Harus Dikunci dengan `for update`?
3.7 Kenapa Nama Fungsi Harus Sama dengan PRD?

Bab 4 · Cara Membaca *Trigger*

4.1 Apa Itu *Trigger*?
4.2 Bagaimana `catat_audit` Mencatat Perubahan?
4.3 Bagaimana `jaga_wo` Membatasi *Update* Langsung?
4.4 Kenapa `jaga_wo` dan `jaga_profil` Tanpa `security definer`?

Bab 5 · Contoh SQL yang Harus Ditolak

5.1 Fungsi Tanpa Pemeriksaan Peran
5.2 *Policy* Tulis dengan `using (true)`
5.3 SQL yang Mengubah atau Menghapus Tabel
5.4 Fungsi Stok Tanpa `for update`
5.5 Nama Fungsi Berbeda dari PRD
5.6 Tanda Lain yang Perlu Ditanyakan

Bab 6 · Langkah Setelah SQL Disetujui

6.1 Menyimpan SQL ke Folder `migrations`
6.2 Cara Memperbarui Tipe `database.ts`
6.3 Cara Menjalankan Advisor Keamanan
6.4 Menguji Fungsi dengan Meniru Login Akun
6.5 Bila Hasil Uji Tidak Sesuai

Lampiran

A Daftar Istilah
B Daftar Pustaka

BAB 1

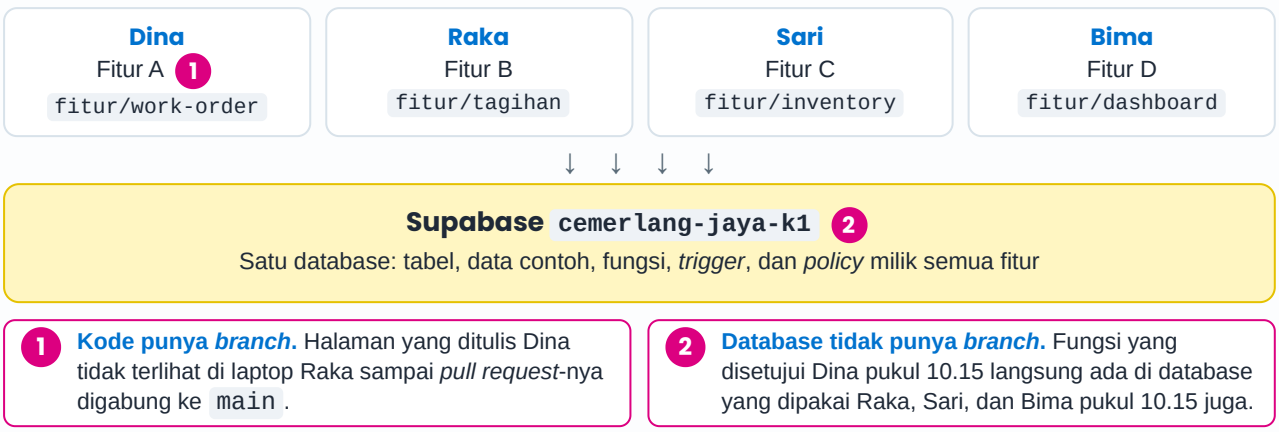
Kenapa SQL dari Agen Perlu Dibaca?

Di Sesi 9, agen AI menulis SQL untuk database aplikasi bengkel lewat Supabase MCP, lalu menunggu persetujuanmu. Tombol setuju terlihat sepele, tetapi di baliknya ada database yang dipakai bersama satu kelompok dan perubahan yang langsung berlaku. Bab ini menjelaskan kenapa SQL itu perlu dibaca dulu.

1.1 Satu Database untuk Empat Anggota

Di tim mana pun, kode setiap orang bisa dipisah ke *branch* masing-masing, tetapi database biasanya hanya satu: tim toko online memakai satu database pesanan, tim ojek online memakai satu database perjalanan. Perubahan di database terlihat oleh semua orang saat itu juga.

Kelompokmu bekerja dengan pola yang sama. Bagan berikut memperlihatkan empat anggota yang masing-masing punya *branch*, tetapi semuanya tersambung ke satu proyek Supabase.



Karena itu, aturan database bersama di Sesi 9 berbunyi: sampaikan ke kelompok sebelum menyetujui SQL, simpan SQL ke `supabase/migrations/fitur_<nama>.sql`, dan jangan mengubah tabel. Kabar disampaikan langsung di kelas, misalnya "Aku mau menyetujui fungsi tagihan, ya."

1.2 Kesalahan yang Sering Ada di SQL Agen

SQL dari agen biasanya berjalan tanpa *error*, tetapi belum tentu aman. Bagian yang tidak tertulis jelas di permintaan diisi agen dengan tebakan, dan tebakan termudah sering kali yang paling longgar. Tabel berikut merangkum kesalahan yang umum ditemui pada SQL buatan agen, beserta akibatnya di aplikasi bengkel.

Kesalahan	Contoh di aplikasi bengkel	Akibat
Lupa memeriksa peran	Fungsi `catat_stok_masuk` langsung menambah stok tanpa melihat siapa pemanggilnya.	Pak Joko (Kepala Mekanik) bisa menambah stok, padahal PRD hanya mengizinkan Admin dan Owner.
Memakai `using (true)` untuk menulis	*Policy update* di `work_orders` diberi syarat `true` agar "tidak ada *error* izin".	Siapa pun yang login bisa mengubah WO tim lain.
Mengubah tabel	Agen menambah kolom atau mengganti nama kolom `stok` agar cocok dengan kodenya.	`seed.sql`, `dashboard.sql`, dan halaman teman yang memakai nama lama menjadi *error*.

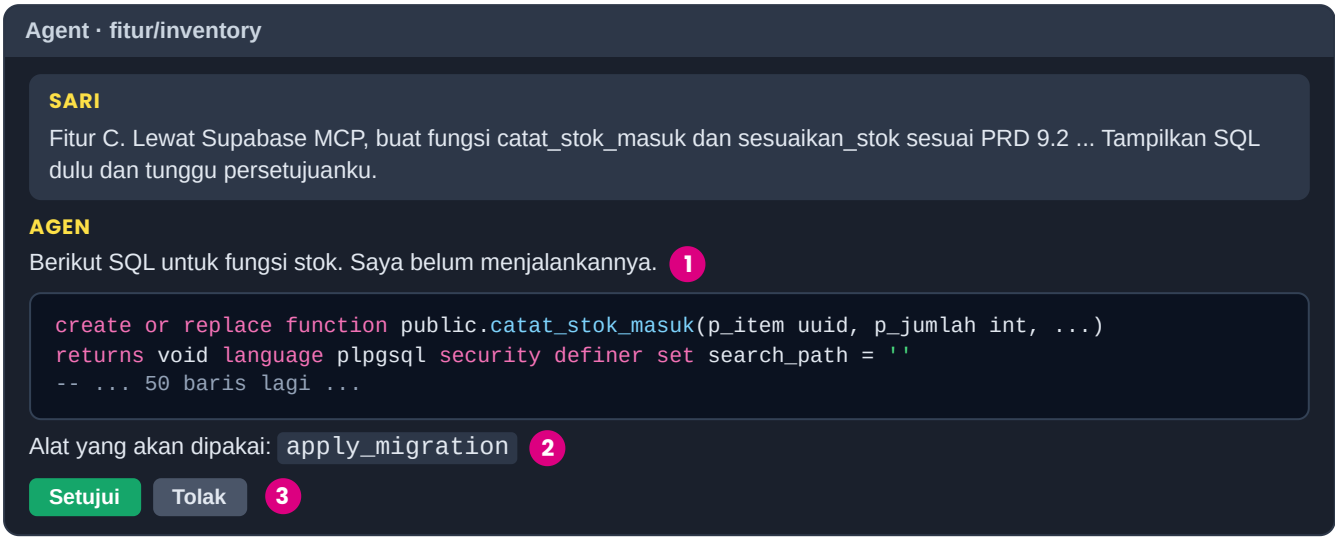
Tidak mengunci baris	Fungsi tagihan membaca stok lalu menguranginya tanpa `for update`.	Dua permintaan bersamaan sama-sama lolos, stok tercatat salah.
Nama berbeda dari PRD	Agen menamai fungsi `tambah_tagihan_item`, bukan `tambah_item_tagihan`.	Halaman yang mengikuti PRD gagal memanggil fungsi.

Kelima kesalahan ini tidak menimbulkan *error* saat SQL dijalankan. Akibatnya baru terlihat saat aplikasi dipakai akun lain, saat dua orang bekerja bersamaan, atau saat teman menjalankan halamannya.

1.3 Kenapa SQL yang Dijalankan Tidak Bisa Ditarik?

Supabase MCP menyediakan dua alat untuk mengubah database. `apply_migration` menerapkan perubahan struktur, seperti membuat fungsi atau *trigger*. `execute_sql` menjalankan perintah SQL biasa, seperti membaca atau mengisi data. Keduanya bekerja langsung pada proyek Supabase kelompok, tanpa tombol *undo*: fungsi tetap ada sampai diganti SQL lain, dan data yang terhapus tidak kembali.

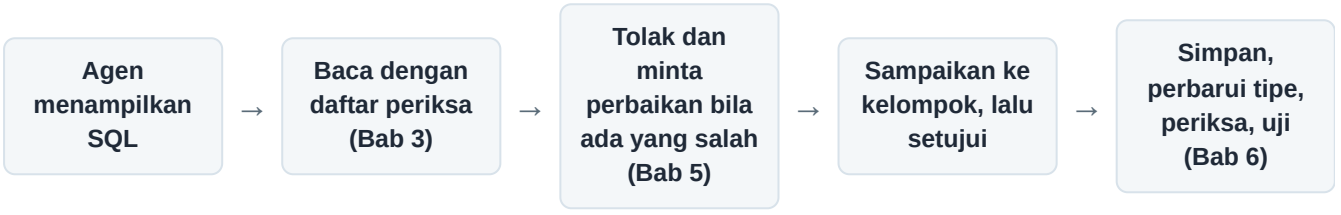
Dokumentasi Supabase MCP sendiri menyarankan persetujuan manual tetap dinyalakan saat bekerja interaktif dan setiap pemanggilan alat diperiksa sebelum dijalankan. Di Antigravity, agen menampilkan SQL di panel Agent lalu menunggu jawabanmu. Ilustrasi berikut meniru tampilan itu saat Sari mengirim Prompt C1.



1. **SQL belum dijalankan.** Database masih sama seperti sebelumnya. Inilah saat terbaik untuk membaca dan bertanya.
2. **Nama alat.** `apply_migration` berarti struktur database akan berubah untuk seluruh kelompok.
3. **Tombol persetujuan.** Ilustrasi disederhanakan; label tombol di Antigravity bisa berbeda. Setelah disetujui, SQL langsung berlaku. Bila ragu, tolak dan minta penjelasan.

1.4 Urutan Membaca dan Menyetujui SQL

Membaca SQL tidak berarti memahami setiap baris seperti programmer database, melainkan mengikuti urutan kerja yang tetap. Bagan berikut memperlihatkan urutan di Sesi 9.



Tabel, RLS *policy*, dan *key* Supabase sudah dibahas di Sesi 8 Modul 4 *Mengenal Supabase*; *transaction*, *race condition*, dan *state machine* di Modul 3 *Menjaga Data Tetap Benar*. Bila SQL terlalu panjang, minta agen merangkumnya, lalu cocokkan dengan SQL aslinya.

BAB 2

Cara Membaca *Database Function*

Sebagian besar SQL yang ditulis agen di Sesi 9 berbentuk *database function*. Setelah satu fungsi dibaca baris demi baris, fungsi lain di aplikasi bengkel akan terasa akrab karena polanya sama.

2.1 Apa Itu *Database Function*?

> **DEFINISI**
> ***Database function*** adalah sekumpulan perintah SQL yang diberi nama dan disimpan di dalam database. Fungsi bisa menerima masukan (parameter), menjalankan beberapa langkah, lalu mengembalikan hasil. Aplikasi cukup memanggil namanya.

Pola ini dipakai di banyak aplikasi. Aplikasi *mobile banking* memproses transfer dengan satu fungsi yang memeriksa saldo, mengurangi saldo pengirim, menambah saldo penerima, dan mencatat mutasi. Toko online memproses *checkout* dengan fungsi yang memeriksa stok, mengurangi stok, dan membuat pesanan. Semua langkah berjalan di database, dekat dengan datanya, sehingga tidak ada langkah yang bisa dilewati dari *browser*.

PRD aplikasi bengkel menetapkan aturan yang sama: semua perubahan penting (status WO, stok, tagihan, *invoice*, pembayaran) ditulis sebagai fungsi PostgreSQL, dan satu panggilan fungsi berjalan sebagai satu *transaction*. Bila salah satu langkah gagal, semua langkah di panggilan itu dibatalkan. Hak *update* langsung ke kolom seperti `stok` dan `status` dicabut dari pengguna, sehingga fungsi menjadi satu-satunya pintu untuk mengubahnya.

2.2 Bagaimana Aplikasi Memanggil Fungsi?

> **DEFINISI**
> `supabase.rpc()` adalah perintah di pustaka JavaScript Supabase untuk memanggil *database function*. Dokumentasi Supabase menjelaskannya sebagai "*perform a function call*". Argumen pertama adalah nama fungsi, argumen kedua adalah objek berisi nilai parameter. RPC adalah singkatan *remote procedure call*.

Contoh berikut memperlihatkan bagaimana halaman `/inventory` milik Sari memanggil `catat_stok_masuk` saat Rina menyimpan formulir Stok Masuk.

```
const { error } = await supabase.rpc('catat_stok_masuk', {     1
  p_item: item.id,                                              2
  p_jumlah: 10,
  p_keterangan: 'Kiriman pemasok',
});
if (error) toast.error(error.message);                          3
```

1. **Nama fungsi.** Harus sama persis dengan nama di database. Satu huruf berbeda, panggilan gagal.
2. **Nama parameter.** Kunci objek dicocokkan dengan nama parameter fungsi, bukan urutannya. `p_item` di sini harus sama dengan `p_item` di SQL.
3. **Pesan penolakan.** Bila fungsi menolak, `error.message` berisi pesan yang ditulis di SQL, misalnya "Jumlah minimal 1". Karena pesannya berbahasa Indonesia, halaman cukup menampilkannya apa adanya.

Contoh ini juga menunjukkan kenapa SQL dan halaman harus sepakat soal nama. Halaman ditulis berdasarkan PRD dan berkas tipe `src/types/database.ts`; fungsi ditulis agen berdasarkan permintaan. Bila keduanya tidak sama, tidak ada yang memberi tahu sampai tombol diklik.

2.3 Bedah Fungsi `catat_stok_masuk`

Fungsi `catat_stok_masuk` dari `fitur_c_inventory.sql` mencatat spare part yang datang dari pemasok. Fungsinya pendek, tetapi memuat hampir semua unsur yang perlu dikenali. Kode berikut adalah isi asli fungsi itu.

Barisnya dipecah agar mudah dibaca, tanpa mengubah perintahnya. Setiap bagian diberi nomor yang dijelaskan di bawahnya.

```
create or replace function public.catat_stok_masuk(                    1
    p_item uuid, p_jumlah int, p_keterangan text default null)         2
returns void                                                           3
language plpgsql                                                       4
security definer                                                       5
set search_path = ''                                                   6
as $$
declare v_peran text := private.peran_saya();                          7
        i public.inventory_items%rowtype;
begin                                                                  8
  if v_peran not in ('admin', 'owner') or v_peran is null then         9
    raise exception 'Hanya Admin atau Owner yang boleh mencatat stok masuk';
  end if;
  select * into i from public.inventory_items                          10
    where id = p_item for update;
  if not found or i.jenis <> 'part' then                               11
    raise exception 'Stok masuk hanya untuk spare part'; end if;
  if p_jumlah is null or p_jumlah < 1 then
    raise exception 'Jumlah minimal 1'; end if;
  update public.inventory_items                                        12
    set stok = stok + p_jumlah, updated_at = now() where id = p_item;
  insert into public.stock_movements                                   13
    (item_id, jenis, jumlah, stok_sesudah, keterangan, dicatat_oleh)
  values (p_item, 'masuk', p_jumlah, i.stok + p_jumlah,
    nullif(trim(p_keterangan), ''), auth.uid());
end $$;
```

1. **Kepala fungsi.** `create or replace function` membuat fungsi baru, atau **menimpa** fungsi lama yang namanya sama. Skema `public` membuat fungsi bisa dipanggil aplikasi lewat `supabase.rpc()`.

2. **Parameter.** Tiga masukan beserta tipenya: ID item (`uuid`), jumlah (`int`), dan keterangan (`text`). `default null` berarti keterangan boleh tidak diisi. Awalan `p_` membedakan parameter dari nama kolom.

3. **Hasil.** `returns void` berarti fungsi tidak mengembalikan nilai. Fungsi lain bisa mengembalikan `uuid` (ID baris baru) atau `boolean`.

4. **Bahasa.** `plpgsql` adalah bahasa prosedural PostgreSQL. Bahasa ini menambahkan variabel, `if`, dan perulangan ke SQL biasa.

5. **Hak pemilik.** Fungsi berjalan dengan hak pembuatnya, bukan hak pemanggil. Inilah yang membuat fungsi bisa mengubah kolom `stok` yang tertutup bagi pengguna. Dibahas di subbab 3.1.

6. **Jalur pencarian kosong.** Fungsi tidak menebak skema; setiap tabel ditulis lengkap, misalnya `public.inventory_items`. Dibahas di subbab 3.2.

7. **Variabel.** Bagian `declare` menyiapkan `v_peran` (langsung diisi peran pemanggil dari `private.peran_saya()`) dan `i` (tempat satu baris item, `%rowtype` berarti bentuknya sama dengan baris tabel).

8. **Isi fungsi.** Semua langkah berada di antara `begin` dan `end`. Tanda `$$` menandai awal dan akhir isi fungsi.

9. **Pemeriksaan peran.** Langkah pertama, sebelum membaca atau mengubah apa pun. Selain Admin dan Owner, termasuk akun nonaktif yang perannya `null`, ditolak.

10. **Baca dan kunci.** Baris item dibaca ke variabel `i` sekaligus dikunci dengan `for update`, sehingga permintaan lain untuk item yang sama menunggu.

11. **Validasi.** `found` bernilai salah bila item tidak ada. Item jasa ditolak karena tidak punya stok. Jumlah harus 1 atau lebih. Setiap penolakan memakai `raise exception` dengan pesan berbahasa Indonesia.

12. **Ubah stok.** Stok ditambah dari nilai di database (`stok + p_jumlah`), bukan dari angka kiriman halaman.

13. **Catat pergerakan.** Satu baris masuk ke `stock_movements` berisi jenis `masuk`, jumlah, stok sesudahnya, keterangan (kosong diubah jadi `null` oleh `nullif(trim(...), '')`), dan siapa yang mencatat (`auth.uid()`, ID pengguna yang login).

2.4 Apa yang Terjadi Saat Fungsi Menolak?

DEFINISI

`raise exception` adalah perintah PL/pgSQL untuk menghentikan fungsi dengan *error*. Dokumentasi PostgreSQL menyebut bahwa `raise exception` biasanya membatalkan *transaction* yang sedang berjalan. Tanda `%` di dalam pesan diganti dengan nilai berikutnya, misalnya `'Stok % hanya %', i.nama, i.stok` menjadi "Stok Lampu H4 hanya 0".

Karena satu panggilan `rpc` adalah satu *transaction*, penolakan di tengah jalan tidak meninggalkan perubahan setengah jadi. Tabel berikut memperlihatkan beberapa panggilan `catat_stok_masuk` untuk Busi Iridium (stok 24) dan hasilnya.

Pemanggil	Masukan	Hasil
Pak Joko	Busi Iridium, 10	**Ditolak** di langkah 9 : "Hanya Admin atau Owner yang boleh mencatat stok masuk". Stok tetap 24.
Rina	Busi Iridium, 0	**Ditolak** di langkah 11 : "Jumlah minimal 1". Kunci baris dilepas, stok tetap 24.
Rina	Ganti Oli (jasa), 5	**Ditolak**: "Stok masuk hanya untuk spare part".
Rina	Busi Iridium, 10, "Kiriman pemasok"	**Berhasil.** Stok menjadi 34, satu baris `masuk` tercatat di `stock_movements` dengan `stok_sesudah` 34, dan *trigger* `catat_audit` mencatat keduanya di `audit_logs`.

Urutan pemeriksaan ikut menentukan hasil. Pak Joko ditolak sebelum fungsi menyentuh tabel item, sehingga ia bahkan tidak tahu apakah item yang dikirim ada atau tidak. Inilah alasan peran diperiksa paling awal.

2.5 Siapa yang Boleh Menjalankan Fungsi?

Di bagian bawah setiap berkas fitur ada blok yang mengatur hak menjalankan fungsi. Potongan berikut diambil dari `fitur_c_inventory.sql`.

```
do $$
declare f text;
begin
  foreach f in array array['catat_stok_masuk(uuid, int, text)', 'sesuaikan_stok(uuid, int, text)']
loop
    execute format('revoke execute on function public.%s from public, anon', f);
    execute format('grant execute on function public.%s to authenticated', f);
  end loop;
end $$;
```

Dokumentasi PostgreSQL menjelaskan bahwa fungsi baru secara bawaan boleh dijalankan oleh `PUBLIC`, yaitu semua peran database. Dokumentasi Supabase menambahkan bahwa tanpa pembatasan, fungsi bisa dipanggil peran mana pun, termasuk `anon` (pengunjung yang belum login). Blok di atas mencabut hak itu dari `public` dan `anon`, lalu memberikannya hanya ke `authenticated` (pengguna yang sudah login). Pemeriksaan peran di dalam fungsi tetap dibutuhkan, karena Pak Joko juga termasuk `authenticated`.

TANDA BERHASIL

Setiap fungsi baru dari agen diikuti blok `revoke ... from public, anon` dan `grant ... to authenticated` yang menyebut nama dan tipe parameter fungsi itu dengan tepat.

BAB 3

Empat Hal di Daftar Periksa SQL

Daftar periksa SQL di Daftar Prompt App 4 berisi empat hal: fungsi aman (`security definer` dan `search_path` kosong), peran diperiksa di awal, baris dikunci dengan `for update`, dan nama sama dengan PRD. Bab ini menjelaskan alasan di balik setiap hal, lengkap dengan potongan SQL aslinya.

3.1 Apa Arti `security definer`?

> **DEFINISI**
> **`security definer`** membuat fungsi berjalan dengan hak pemilik fungsi. Lawannya, `security invoker`, membuat fungsi berjalan dengan hak pemanggil dan merupakan pilihan bawaan PostgreSQL. Fungsi yang dibuat lewat Dashboard atau *migration* Supabase dimiliki peran `postgres`, yang tidak dibatasi RLS.

Di aplikasi bengkel, pengguna sengaja tidak diberi hak mengubah kolom `stok` dan `status`. Agar perubahan tetap bisa terjadi lewat jalur resmi, fungsi PRD 9.2 dijalankan dengan hak pemiliknya. Pola ini juga dipakai di aplikasi lain: aplikasi *mobile banking* tidak mengizinkan nasabah mengubah saldo langsung, tetapi fungsi transfer boleh.

Kekuatan ini punya harga. Dokumentasi Supabase menyarankan `security invoker` sebagai pilihan utama, karena fungsi `security definer` bisa membaca dan mengubah baris yang tidak boleh disentuh pemanggil. Maka setiap fungsi `security definer` wajib membawa dua pengaman: `search_path` kosong dan pemeriksaan peran di awal. Tanpa keduanya, fungsi menjadi jalan pintas yang melewati semua RLS *policy*.

3.2 Kenapa `search_path` Harus Kosong?

> **DEFINISI**
> **`search_path`** adalah daftar skema yang diperiksa PostgreSQL ketika sebuah nama tabel atau fungsi ditulis tanpa skema. Bila fungsi menulis `inventory_items` saja, PostgreSQL mencarinya di skema-skema dalam daftar itu, sesuai urutannya.

Masalahnya, pencarian itu bisa menemukan objek yang salah. Dokumentasi PostgreSQL menjelaskan bahwa fungsi `security definer` perlu mengatur `search_path` agar tidak memuat skema yang bisa ditulisi pengguna yang tidak dipercaya. Bila tidak, orang lain bisa membuat objek dengan nama yang sama di skema yang ikut dicari, dan fungsi akan memakai objek itu dengan hak pemiliknya. Skema sementara (`pg_temp`) disebut secara khusus karena secara bawaan diperiksa paling dulu dan umumnya bisa ditulisi siapa saja.

Supabase memilih cara paling ketat: `set search_path = ''`. Daftar pencarian dikosongkan, sehingga fungsi tidak menebak apa pun. Akibatnya, setiap nama di dalam fungsi wajib ditulis lengkap dengan skemanya. Dua potongan berikut memperlihatkan bedanya.

TOLAK

```
returns void language plpgsql
security definer
as $$
begin
  update inventory_items set ...
```

SETUJUI

```
returns void language plpgsql
security definer set search_path = ''
as $$
begin
  update public.inventory_items set ...
```

Potongan kiri bergantung pada `search_path` milik siapa pun yang memanggilnya. Potongan kanan selalu memakai `public.inventory_items`, tidak peduli siapa pemanggilnya. Saat membaca SQL agen, cari `set search_path = ''` tepat setelah `security definer`, lalu periksa sekilas bahwa nama tabel dan fungsi di dalamnya diawali skema: `public.`, `private.`, atau `auth.`.

CATATAN

Fungsi `security definer` tanpa `search_path` yang dikunci akan ditandai Security Advisor Supabase dengan peringatan *Function Search Path Mutable* (lint 0011). Cara menjalankan advisor dibahas di subbab 6.3.

3.3 Kenapa Peran Diperiksa di Awal?

Karena fungsi `security definer` berjalan sebagai pemilik yang tidak dibatasi RLS, *policy* tabel tidak lagi menjaga apa pun di dalam fungsi. Satu-satunya penjaga adalah pemeriksaan yang ditulis di fungsi itu sendiri. PRD 9.2 menetapkan pemeriksaan peran lewat `private.peran_saya()` di baris pertama.

DEFINISI

`private.peran_saya()` adalah fungsi bantuan dari `01_dasar.sql` yang mengembalikan peran pengguna yang login (`owner`, `admin`, atau `kepala_mekanik`) dari tabel `profiles`, hanya bila akunnya aktif. Untuk akun nonaktif atau orang yang belum login, hasilnya `null`.

Perhatikan lagi baris pemeriksaan di `catat_stok_masuk`:

```
if v_peran not in ('admin', 'owner') or v_peran is null then
  raise exception 'Hanya Admin atau Owner yang boleh mencatat stok masuk';
end if;
```

Bagian `or v_peran is null` sering dihapus agen karena terlihat berlebihan, padahal penting. Di SQL, perbandingan dengan `null` tidak menghasilkan benar atau salah, melainkan `null`. Ungkapan `null not in ('admin', 'owner')` menghasilkan `null`, dan `if` yang menerima `null` tidak menjalankan isinya. Tanpa tambahan itu, akun nonaktif justru lolos pemeriksaan.

Fungsi yang membutuhkan data baris sebelum bisa memeriksa hak, seperti `tambah_item_tagihan`, membaca baris WO lebih dulu, lalu langsung memeriksa peran dan tim. Aturannya tetap sama: **pemeriksaan selesai sebelum ada data yang diubah**.

3.4 Pemeriksaan Tim di `ubah_status_wo`

Memeriksa peran saja belum cukup untuk aksi yang melekat pada WO tertentu. Pak Joko dan Pak Slamet sama-sama Kepala Mekanik, tetapi Pak Joko hanya boleh mengurus WO Tim A. Pemeriksaan ini disebut pemeriksaan hubungan: bukan "apa peranmu", melainkan "apa hubunganmu dengan data ini". Aplikasi lain punya pemeriksaan serupa: pengemudi ojek online hanya boleh menyelesaikan perjalanan yang ia ambil, dan penjual di *marketplace* hanya boleh mengubah pesanan tokonya.

DEFINISI

`private.tim_wo(p_wo)` adalah fungsi bantuan yang mengembalikan `kepala_mekanik_id` sebuah WO, yaitu ID Kepala Mekanik pemilik tim WO itu. Fungsi ini dipakai di *policy*, misalnya *policy* unggah foto pengerjaan: `private.tim_wo(...) = (select auth.uid())`.

Di dalam `ubah_status_wo`, baris WO sudah dibaca dan dikunci ke variabel `w`, sehingga fungsi memeriksa kolom yang sama, `w.kepala_mekanik_id`, secara langsung. Potongan berikut diambil dari bagian awal fungsi itu.

```
select * into w from public.work_orders where id = p_wo for update;      1
if not found then raise exception 'Work order tidak ditemukan'; end if;
if v_peran is null then raise exception 'Akun tidak aktif'; end if;          2
v_km_tim := v_peran = 'owner'                                                3
  or (v_peran = 'kepala_mekanik' and w.kepala_mekanik_id = auth.uid());
```

1. **Baca dan kunci WO.** Data tim baru bisa diperiksa setelah baris WO dibaca.
2. **Tolak akun nonaktif** sebelum pemeriksaan lain.
3. **Pemeriksaan hubungan.** `v_km_tim` bernilai benar hanya untuk Owner, atau Kepala Mekanik yang ID-nya sama dengan `kepala_mekanik_id` WO ini. Hasilnya dipakai di setiap perpindahan status milik Kepala Mekanik.

Akibatnya, bila Pak Joko memanggil `ubah_status_wo` untuk WO Tim B, fungsi menolak dengan pesan "Hanya Kepala Mekanik tim ini yang boleh mengubah status WO ini", walaupun perannya benar. Fungsi `tugaskan_mekanik` menambah satu pemeriksaan lagi: semua mekanik yang ditugaskan harus aktif dan berasal dari tim WO itu, sehingga Pak Slamet tidak bisa menugaskan Andi dari Tim A.

3.5 Aturan Perpindahan Status di Dalam Fungsi

Peta status di PRD 7.2 adalah *state machine* yang sudah dikenal dari Sesi 8. Di SQL, peta itu ditulis sebagai rangkaian `if ... elsif`: setiap cabang mencocokkan pasangan status lama dan status baru, memeriksa pelaku, lalu memeriksa syaratnya. Potongan berikut memperlihatkan tiga cabang dan penutupnya.

```
if w.status = 'menunggu' and p_status = 'dikerjakan' then                    (1)
  if not v_km_tim then raise exception 'Hanya Kepala Mekanik tim ini yang boleh mengubah status WO
ini'; end if;
  if not exists (select 1 from public.work_order_mekanik where work_order_id = p_wo)
     or w.estimasi_selesai is null then
    raise exception 'Tunjuk mekanik dan isi estimasi selesai dulu';          (2)
  end if;
-- ... cabang lain ...
elsif w.status = 'siap_diambil' and p_status = 'diambil' then
  if not v_admin then raise exception 'Hanya Admin atau Owner yang boleh menyerahkan mobil'; end
if;  (3)
  -- invoice lunas dan foto kondisi akhir diperiksa di sini
elsif w.status in ('menunggu', 'dikerjakan', 'tertunda') and p_status = 'dibatalkan' then
  -- alasan wajib, stok spare part dikembalikan
else
  raise exception 'Perpindahan status dari % ke % tidak diizinkan', w.status, p_status;  (4)
end if;
```

(1) **Satu cabang, satu baris di PRD 7.2.** Pasangan `menunggu` ke `dikerjakan` hanya untuk Kepala Mekanik tim WO atau Owner.

(2) **Syarat perpindahan.** Minimal satu mekanik dan estimasi selesai terisi, sesuai kolom Syarat di PRD.

(3) **Pelaku berbeda per cabang.** Menyerahkan mobil adalah pekerjaan Admin atau Owner, bukan Kepala Mekanik.

(4) **Penutup yang menolak.** Semua pasangan yang tidak ada di tabel, misalnya `menunggu` ke `siap_diambil`, jatuh ke `else` dan ditolak.

Saat membaca versi agen, cocokkan jumlah cabangnya dengan jumlah baris tabel PRD 7.2 (delapan baris), lalu pastikan ada `else` yang menolak. Fungsi tanpa `else` semacam ini akan meloloskan perpindahan apa pun yang tidak tercantum, karena setelah rangkaian `if` fungsi langsung mengubah status.

3.6 Kapan Baris Harus Dikunci dengan `for update`?

> **DEFINISI**
>
> `select ... for update` membaca baris sekaligus menguncinya. Menurut dokumentasi PostgreSQL, baris yang dikunci tidak bisa dikunci, diubah, atau dihapus *transaction* lain sampai *transaction* saat ini selesai; *transaction* lain yang mencoba akan menunggu. Kunci dilepas saat *transaction* selesai atau dibatalkan.

Baris perlu dikunci bila nilainya dibaca, diperiksa, lalu diubah berdasarkan hasil pemeriksaan itu, dan baris yang sama bisa diincar dua permintaan sekaligus. Contoh di aplikasi lain: kursi terakhir konser, voucher dengan kuota satu, atau saldo dompet digital. Di aplikasi bengkel, PRD 10.3 menguji kasus ini dengan Lampu H4 yang stoknya tinggal 1.

Pak Joko membuka halaman tagihan WO yang sama di dua tab, lalu menekan Tambah Lampu H4 di kedua tab hampir bersamaan. Tabel berikut membandingkan apa yang terjadi tanpa kunci, pada fungsi yang menghitung stok baru dari angka yang dibacanya (seperti contoh di subbab 5.4), dan dengan kunci.

Tanpa `for update`		
T1	Tab 1 membaca stok: 1	Tab 2 membaca stok: 1
T2	1 ≥ 1, lolos	1 ≥ 1, lolos
T3	Menulis stok 1 − 1 = 0, item dicatat	Menulis stok 1 − 1 = 0, item dicatat

Salah: dua Lampu H4 masuk tagihan padahal hanya satu di rak, dan stok tertulis 0. Batas `stok >= 0` di tabel tidak menolong, karena tidak ada angka minus yang ditulis.

Dengan `for update`		
T1	Tab 1 membaca dan mengunci baris: 1	Tab 2 ingin membaca, **menunggu**
T2	Stok jadi 0, item dicatat, selesai	Kunci lepas, Tab 2 membaca stok: 0
T3	Halaman menampilkan item baru	Ditolak: "Stok Lampu H4 hanya 0"

Benar: satu berhasil, satu ditolak, sesuai PRD 10.3.

Potongan berikut diambil dari `tambah_item_tagihan` di `fitur_b_tagihan.sql`. Fungsi ini mengunci dua baris: baris WO (agar status tidak berubah di tengah jalan) dan baris item (agar stok tidak direbut).

```
select * into w from public.work_orders where id = p_wo for update;          1
-- ... pemeriksaan tim, status dikerjakan, jumlah ...
-- kunci baris item agar dua permintaan bersamaan antre
select * into i from public.inventory_items where id = p_item for update;    2
if not found or not i.aktif then raise exception 'Item tidak ditemukan atau tidak aktif'; end if;
if i.jenis = 'part' then
  if i.stok < p_jumlah then raise exception 'Stok % hanya %', i.nama, i.stok; end if;    3
  update public.inventory_items set stok = stok - p_jumlah, updated_at = now() where id = p_item;
```

1. **Kunci WO.** Bila Admin membatalkan WO bersamaan, salah satu menunggu yang lain selesai.
2. **Kunci item.** Tab kedua berhenti di baris ini sampai tab pertama selesai.
3. **Pemeriksaan stok setelah kunci.** Karena dibaca setelah antre, Tab 2 melihat stok 0 dan ditolak dengan pesan yang menyebut nama item dan sisa stoknya.

Data lain yang perlu dikunci di aplikasi bengkel adalah baris WO pada setiap perubahan status, baris *invoice* saat mencatat pembayaran, dan baris penghitung nomor di `number_counters` (dikunci lewat *upsert* di `private.nomor_berikut`) agar dua WO tidak mendapat nomor yang sama.

3.7 Kenapa Nama Fungsi Harus Sama dengan PRD?

Nama fungsi adalah titik temu antara pekerjaan banyak orang. Di aplikasi bengkel, ada tiga pihak yang bergantung pada nama yang sama:

- **Berkas bersama di starter.** `seed.sql` dan `dashboard.sql` ditulis berdasarkan nama tabel dan kolom PRD. Fungsi dashboard memanggil `private.peran_saya()` dengan nama itu.
- **Halaman teman.** Halaman `/work-orders/[id]` milik Dina menampilkan tombol Serahkan Mobil yang hanya berhasil bila *invoice* dari fitur Raka sudah lunas; keduanya bertemu lewat nama tabel dan fungsi PRD.
- **Berkas tipe.** `src/types/database.ts` dibuat ulang dari database. Bila nama fungsi berubah, editor menandai panggilan `rpc` lama sebagai *error*.

Aturan yang sama berlaku untuk parameter. PRD 9.2 menulis nama parameter lengkap dengan awalan `p_`, misalnya `tambah_item_tagihan(p_wo, p_item, p_jumlah)`. Awalan ini lazim di PL/pgSQL; dokumentasi PostgreSQL menjelaskan bahwa nama yang bisa berarti variabel sekaligus kolom akan menimbulkan *error*, dan cara paling sederhana adalah memakai pola nama yang berbeda. Nama parameter wajib sama persis karena `supabase.rpc()` mencocokkan argumen berdasarkan namanya: halaman memanggil `rpc('tambah_item_tagihan', { p_wo, p_item, p_jumlah })`.

TANDA BERHASIL

Nama fungsi dan nama parameter di SQL agen sama huruf demi huruf dengan tabel PRD 9.2, dan tipe hasilnya sama (misalnya `buat_work_order` mengembalikan `uuid`).

BAB 4

Cara Membaca *Trigger*

Selain fungsi yang dipanggil aplikasi, agen juga menulis fungsi yang berjalan sendiri setiap kali data berubah. Fungsi jenis ini dipasang sebagai *trigger*. Fitur A memasang `jaga_wo`, Fitur D memasang `catat_audit`, dan starter sudah membawa `jaga_profil`.

4.1 Apa Itu *Trigger*?

> **DEFINISI**
> ***Trigger*** adalah aturan di database yang menjalankan sebuah fungsi secara otomatis setiap kali terjadi operasi tertentu (`insert` , `update` , atau `delete`) pada sebuah tabel. Fungsi yang dijalankan disebut fungsi *trigger* dan ditandai `returns trigger` .

Contohnya: aplikasi HR mencatat riwayat setiap kali gaji diubah, dan toko online mengisi waktu "terakhir diubah" setiap kali produk disunting. Tidak ada halaman yang memanggilnya. Tabel berikut merangkum kata kunci yang selalu muncul di SQL *trigger*.

Kata kunci	Arti
`before update`	Fungsi berjalan **sebelum** baris diubah. Cocok untuk memeriksa atau merapikan data, dan bisa menolak perubahan dengan `raise exception` .
`after insert or update or delete`	Fungsi berjalan **setelah** perubahan terjadi. Cocok untuk mencatat ke tabel lain, karena nilai akhirnya sudah pasti.
`for each row`	Fungsi dijalankan sekali untuk setiap baris yang berubah.
`new` dan `old`	`new` berisi baris versi baru (pada `insert` dan `update`), `old` berisi versi lama (pada `update` dan `delete`).
`tg_op`	Jenis operasi yang memicu: `INSERT` , `UPDATE` , atau `DELETE` .

Fungsi *trigger* `before` mengembalikan `new` agar perubahan dilanjutkan. Dokumentasi PostgreSQL menjelaskan bahwa bila *trigger* `before` per baris mengembalikan `null` , operasi untuk baris itu dibatalkan tanpa pesan. Sebaliknya, nilai kembalian *trigger* `after` diabaikan, sehingga boleh `null` .

4.2 Bagaimana `catat_audit` Mencatat Perubahan?

PRD 6.13 meminta Audit Log: Owner bisa melihat siapa mengubah apa, kapan, dan nilai sebelum serta sesudahnya. Bima memasang *trigger* `catat_audit` di sembilan tabel penting. Berikut isi fungsinya dari `fitur_d_audit.sql` .

```
create or replace function private.catat_audit()
returns trigger language plpgsql security definer set search_path = ''          1
as $$
declare v_new jsonb := case when tg_op <> 'DELETE' then to_jsonb(new) end;      2
        v_old jsonb := case when tg_op <> 'INSERT' then to_jsonb(old) end;
        v_row jsonb := coalesce(v_new, v_old);
begin
  if tg_op = 'UPDATE' and (v_new - 'updated_at') = (v_old - 'updated_at') then  3
    return null; end if;
  insert into public.audit_logs (pelaku_id, pelaku_peran, aksi, tabel, data_id,      4
                                 sebelum, sesudah, alasan)
  values (auth.uid(), (select role from public.profiles where id = auth.uid()),
          tg_op, tg_table_name,
          coalesce(v_row->>'nomor', v_row->>'kode', v_row->>'id', v_row->>'item_id', v_row-
>>'billing_item_id'),
          v_old, v_new,
          coalesce(v_new->>'alasan_penyesuaian', v_new->>'catatan_status',
                   v_new->>'alasan_diskon', v_new->>'keterangan'));
  return null;
end $$;
```

1. **`security definer` di sini disengaja.** Pengguna hanya punya hak membaca `audit_logs`, tidak menulis. Fungsi butuh hak pemilik untuk menambah baris log.
2. **Salinan baris.** `to_jsonb` mengubah baris lama dan baru menjadi JSON. Saat `INSERT` tidak ada versi lama; saat `DELETE` tidak ada versi baru.
3. **Lewati perubahan kosong.** Bila yang berubah hanya `updated_at`, tidak ada yang dicatat.
4. **Isi log.** Pelaku dan perannya, jenis aksi, nama tabel, nomor atau ID data, nilai sebelum dan sesudah, serta alasan bila ada.

Fungsi itu dipasang ke sembilan tabel PRD 9.4 dengan `create trigger catat_audit after insert or update or delete ... for each row`. Bila fungsi bisnis menolak dengan `raise exception`, seluruh *transaction* dibatalkan dan baris log ikut batal.

4.3 Bagaimana `jaga_wo` Membatasi *Update* Langsung?

Tidak semua perubahan WO lewat fungsi. Rina boleh membetulkan keluhan atau odometer selama WO masih `menunggu`, dan Kepala Mekanik tim boleh mengisi diagnosis saat WO `dikerjakan` atau `tertunda`. Hak kolom di `01_dasar.sql` hanya membuka empat kolom itu, dan *policy* `wo_ubah` menentukan baris mana yang boleh disentuh. Yang belum dijaga adalah kombinasi kolom, peran, dan status. Bagian itu dijaga *trigger* `jaga_wo`.

```
create or replace function private.jaga_wo()
returns trigger language plpgsql set search_path = ''                          1
as $$
declare v_peran text; v_ubah_masuk boolean; v_ubah_diag boolean;
begin
  if current_user <> 'authenticated' then return new; end if;   -- dari fungsi, lewati   2
  v_peran := private.peran_saya();
  v_ubah_masuk := new.keluhan is distinct from old.keluhan                          3
                  or new.odometer is distinct from old.odometer
                  or new.kepala_mekanik_id is distinct from old.kepala_mekanik_id;
  v_ubah_diag := new.diagnosis is distinct from old.diagnosis;
  if v_ubah_masuk and not (v_peran in ('admin', 'owner') and old.status = 'menunggu') then   4
    raise exception 'Data masuk hanya bisa diubah Admin atau Owner saat status menunggu';
  end if;
  -- pemeriksaan diagnosis: Kepala Mekanik tim WO atau Owner, saat dikerjakan atau tertunda
  return new;                                                                       5
end $$;
create trigger jaga_wo before update on public.work_orders
for each row execute function private.jaga_wo();
```

1. **Tidak ada `security definer`.** Fungsi berjalan dengan hak siapa pun yang memicunya. Alasannya di subbab 4.4.

2. **Siapa yang mengubah?** Bila perubahan tidak datang langsung dari pengguna aplikasi, *trigger* melepasnya.

3. **Kolom apa yang berubah?** `is distinct from` membandingkan nilai lama dan baru, termasuk saat salah satunya `null`.

4. **Cocokkan dengan PRD 10.2.** Data masuk hanya Admin atau Owner saat `menunggu`. Pak Joko yang mencoba mengubah keluhan ditolak.

5. **Lanjutkan perubahan.** `return new` berarti perubahan boleh disimpan. Bila agen menulis `return null` di *trigger* `before`, setiap perubahan akan dibatalkan diam-diam tanpa pesan.

Trigger `jaga_profil` di starter bekerja dengan pola yang sama pada tabel `profiles`: bila perubahan datang dari pengguna aplikasi, kolom `role`, `aktif`, dan `email` hanya boleh diubah Owner. Ada satu aturan tambahan yang berlaku untuk siapa pun: Owner terakhir yang aktif tidak boleh diturunkan atau dinonaktifkan, agar bengkel tidak kehilangan satu-satunya akun yang bisa mengatur pengguna.

4.4 Kenapa `jaga_wo` dan `jaga_profil` Tanpa `security definer`?

Jawabannya ada di `current_user`, nilai yang dibaca kedua *trigger* itu di baris pertamanya.

> **DEFINISI**
>
> `current_user` adalah nama peran database yang sedang dipakai untuk memeriksa hak. Dokumentasi PostgreSQL menyebut bahwa nilainya berubah selama fungsi `security definer` berjalan: di dalam fungsi seperti itu, `current_user` adalah pemilik fungsi.

Di Supabase, permintaan dari aplikasi oleh pengguna yang login berjalan sebagai peran `authenticated`. Dengan bekal itu, ada dua jalur perubahan WO yang bisa dibedakan:

Jalur perubahan	`current_user` di *trigger*	Tindakan `jaga_wo`
Rina mengubah keluhan lewat halaman (*update* langsung)	`authenticated`	Memeriksa kolom, peran, dan status.
Pak Joko memanggil `ubah_status_wo` lewat `rpc`	pemilik fungsi (`postgres`)	Dilewati, karena fungsi itu sudah memeriksa peran, tim, dan perpindahan status sendiri.

Pembedaan ini hanya berhasil bila *trigger* sendiri berjalan dengan hak pemicunya. Bila agen menambahkan `security definer` ke `jaga_wo`, `current_user` di dalam *trigger* akan selalu bernilai pemilik fungsi, dari jalur mana pun perubahan datang. Syarat `current_user <> 'authenticated'` selalu terpenuhi, *trigger* selalu langsung mengembalikan `new`, dan semua pemeriksaannya tidak pernah dijalankan. Untuk `jaga_profil`, akibatnya lebih serius: Pak Joko bisa mengubah perannya sendiri menjadi `owner`, persis uji yang wajib ditolak di PRD 10.3. Catatan di PRD 9.4 menegaskan hal ini: kedua fungsi *trigger* itu ditulis tanpa `security definer`.

> **PERHATIKAN**
>
> Aturan "fungsi harus `security definer`" berlaku untuk fungsi PRD 9.2 dan fungsi seperti `catat_audit` yang menulis ke tabel tertutup. Fungsi *trigger* yang memeriksa `current_user` justru tanpa `security definer`. Semuanya tetap memakai `set search_path = ''`.

BAB 5

Contoh SQL yang Harus Ditolak

Menolak SQL bukan berarti gagal. Penolakan adalah bagian biasa dari bekerja dengan agen: kamu menunjuk bagian yang salah, menyebut aturan yang dilanggar, dan meminta versi perbaikan untuk dibaca lagi. Bab ini memuat lima contoh SQL yang salah, alasan penolakannya, dan kalimat permintaan perbaikan yang bisa dikirim ke agen.

5.1 Fungsi Tanpa Pemeriksaan Peran

Potongan berikut tampak lengkap: ada `security definer`, ada `search_path` kosong, dan stok bertambah dengan benar. Yang hilang adalah pemeriksaan siapa pemanggilnya.

TOLAK · catat_stok_masuk tanpa pemeriksaan peran

```
returns void language plpgsql security definer set search_path = ''
as $$
begin
  update public.inventory_items set stok = stok + p_jumlah where id = p_item;
  -- tidak ada private.peran_saya(), tidak ada raise exception
```

KENAPA DITOLAK

Fungsi berjalan dengan hak pemilik dan melewati RLS, sementara hak `execute` diberikan ke semua pengguna `authenticated`. Artinya Pak Joko, Pak Slamet, bahkan akun nonaktif bisa menambah stok sesuka hati. PRD 9.2 hanya mengizinkan Admin dan Owner.

CONTOH PERMINTAAN PERBAIKAN

Fungsi catat_stok_masuk belum memeriksa peran. Tambahkan pemeriksaan di baris pertama lewat private.peran_saya(): hanya admin dan owner yang boleh, dan peran null (akun nonaktif) juga ditolak. Pakai raise exception dengan pesan berbahasa Indonesia sesuai PRD 9.2. Tampilkan SQL perbaikannya dulu dan tunggu persetujuanku.

5.2 *Policy* Tulis dengan `using (true)`

Agen kadang menulis *policy* paling longgar ketika menemui *error* izin saat mencoba halaman. Potongan berikut membuat *error* itu hilang, sekaligus membuka seluruh tabel.

TOLAK · policy update terbuka

```
create policy "wo_ubah_semua" on public.work_orders for update to authenticated
  using (true) with check (true);
```

KENAPA DITOLAK

`using (true)` berarti syaratnya selalu terpenuhi untuk setiap baris. RLS *policy* bersifat menambah izin: bila satu *policy* mengizinkan, *policy* lain yang lebih ketat tidak lagi berarti. Pak Joko bisa mengubah WO Tim B. PRD 10.1 tegas melarang `using (true)` untuk operasi tulis. Supabase Security Advisor juga punya pemeriksaan untuk *policy* yang terlalu longgar.

CONTOH PERMINTAAN PERBAIKAN

Hapus policy wo_ubah_semua. PRD 10.1 melarang using (true) untuk operasi tulis, dan policy wo_ubah dari 01_dasar.sql sudah mengatur update work_orders. Jelaskan dulu error izin apa yang tadi muncul dan dari halaman mana, supaya kita cari penyebab sebenarnya. Jangan menjalankan apa pun sebelum aku setujui.

CATATAN

`using (true)` untuk **membaca** data yang memang publik tidak termasuk larangan ini. Contohnya *policy* `setelan_baca` di starter, karena nama, alamat, dan jam buka bengkel memang tampil di halaman depan.

5.3 SQL yang Mengubah atau Menghapus Tabel

Tabel aplikasi bengkel sudah lengkap di `01_dasar.sql`. Meski begitu, agen bisa mengusulkan perubahan tabel agar kodenya lebih mudah, misalnya menambah kolom bantuan atau mengganti nama kolom.

TOLAK · mengubah struktur tabel

```
alter table public.work_orders add column nama_mekanik text;
alter table public.inventory_items rename column stok to jumlah_stok;
drop table if exists public.billing_items cascade;
```

KENAPA DITOLAK

Tabel dipakai bersama. Mengganti nama `stok` membuat fungsi Raka, halaman Sari, dan `dashboard.sql` rusak bersamaan. `drop table ... cascade` menghapus tabel beserta isinya dan objek yang bergantung padanya, dan data yang hilang tidak bisa dikembalikan lewat *undo*. Daftar periksa SQL dan `AGENTS.md` sama-sama melarang membuat, mengubah, atau menghapus tabel.

CONTOH PERMINTAAN PERBAIKAN

Tabel tidak boleh diubah karena dipakai bersama satu kelompok dan sudah lengkap di 01_dasar.sql. Batalkan rencana alter table dan drop table. Penuhi kebutuhan ini dengan tabel dan kolom yang sudah ada sesuai PRD Bab 8, misalnya nama mekanik diambil lewat work_order_mekanik dan mekanik. Tampilkan pendekatan barunya dulu.

5.4 Fungsi Stok Tanpa `for update`

Potongan berikut memeriksa stok dengan benar dalam kondisi satu pengguna. Kesalahannya baru muncul saat dua permintaan datang bersamaan.

TOLAK · baca stok tanpa kunci

```
select stok into v_stok from public.inventory_items where id = p_item;
if v_stok < p_jumlah then raise exception 'Stok tidak cukup'; end if;
update public.inventory_items set stok = v_stok - p_jumlah where id = p_item;
```

KENAPA DITOLAK

Ada dua masalah. Pertama, tanpa `for update`, dua tab bisa membaca stok Lampu H4 yang sama-sama 1 dan sama-sama lolos (subbab 3.6). Kedua, stok baru dihitung dari angka lama di variabel (`v_stok - p_jumlah`), sehingga pengurangan salah satu tab bisa tertimpa. Ini adalah *race condition* dari Sesi 8 Modul 3.

CONTOH PERMINTAAN PERBAIKAN

Fungsi ini membaca stok tanpa mengunci baris. Ubah menjadi select ... for update pada baris inventory_items sebelum stok diperiksa, lalu kurangi stok dengan set stok = stok - p_jumlah. Pesan penolakannya ikuti PRD: 'Stok % hanya %' dengan nama item dan sisa stok. Tampilkan SQL perbaikannya dulu.

5.5 Nama Fungsi Berbeda dari PRD

Kesalahan ini paling sering lolos karena SQL-nya sendiri benar. Yang salah hanya namanya.

TOLAK · nama dan parameter tidak sesuai PRD 9.2

```
create or replace function public.tambah_tagihan_item(jumlah int, item uuid, wo uuid)
returns void ...
```

KENAPA DITOLAK

PRD 9.2 menulis `tambah_item_tagihan(p_wo, p_item, p_jumlah)` yang mengembalikan `uuid`. Di potongan ini namanya tertukar, urutan parameternya terbalik, dan hasilnya `void`. Halaman yang memanggil `rpc('tambah_item_tagihan', ...)` akan gagal karena fungsi itu tidak ada. Selain itu, nama kolom `jumlah` di tabel sama dengan nama parameter, sehingga fungsi berisiko *error* nama ganda.

CONTOH PERMINTAAN PERBAIKAN

Nama, urutan parameter, dan tipe hasil fungsi ini tidak sama dengan PRD 9.2. Ganti menjadi tambah_item_tagihan dengan parameter WO, item, lalu jumlah, memakai awalan p_ seperti fungsi lain di proyek ini (p_wo, p_item, p_jumlah), dan returns uuid berisi ID baris billing_items yang baru. Pastikan tidak ada fungsi lain dengan nama lama yang tertinggal. Tampilkan SQL-nya dulu.

5.6 Tanda Lain yang Perlu Ditanyakan

Selain lima contoh di atas, ada beberapa tanda yang belum tentu salah, tetapi perlu ditanyakan ke agen sebelum menyetujui. Tabel berikut merangkumnya beserta pertanyaan yang bisa dikirim.

Yang terlihat di SQL	Kenapa perlu ditanyakan	Pertanyaan ke agen
`create or replace function` dengan nama milik fitur lain	`or replace` menimpa fungsi yang sudah ada tanpa peringatan. Fungsi teman bisa hilang.	"Fungsi ini milik fitur siapa menurut PRD 15.4? Kenapa perlu diganti?"
`grant execute ... to anon`	Pengunjung yang belum login bisa memanggil fungsi. Tidak ada fungsi PRD 9.2 yang butuh ini.	"Kenapa anon perlu hak execute? Cabut bila tidak ada di PRD."
`disable row level security`	Mematikan semua *policy* di tabel itu. PRD 10.1 mewajibkan RLS menyala di semua tabel.	"Kembalikan RLS menyala dan jelaskan masalah yang ingin diatasi."
Trigger yang memeriksa `current_user` diberi `security definer`	Pemeriksaannya tidak akan pernah berjalan (subbab 4.4).	"Hapus security definer dari fungsi trigger ini sesuai catatan PRD 9.4."
`delete from` atau `update` tanpa `where`	Mengenai semua baris di tabel.	"Baris mana yang dimaksud? Tambahkan syarat where."

Setelah agen memperbaiki SQL, baca ulang dengan daftar periksa yang sama. Perbaikan satu bagian kadang mengubah bagian lain, misalnya pemeriksaan peran ditambahkan tetapi `search_path` ikut hilang.

TANDA BERHASIL

SQL versi akhir lolos keempat hal di daftar periksa, tidak memuat `alter table`, `drop table`, atau *policy* tulis `using (true)`, dan kelompok sudah mendengar bahwa SQL ini akan disetujui.

BAB 6

Langkah Setelah SQL Disetujui

Persetujuan bukan akhir pekerjaan. Setelah SQL berhasil dijalankan, masih ada empat langkah: menyimpan SQL ke repo, memperbarui berkas tipe, menjalankan advisor keamanan, dan menguji fungsi dengan meniru login akun uji.

6.1 Menyimpan SQL ke Folder `migrations`

DEFINISI

Migration adalah berkas berisi perubahan struktur database, disimpan di repo agar perubahan itu tercatat, bisa dibaca orang lain, dan bisa dijalankan ulang di database baru.

Database tidak punya *branch*, sehingga satu-satunya jejak perubahan database di GitHub adalah berkas SQL di repo. Prompt A1, B1, C1, dan D1 sudah meminta agen menyimpan SQL yang disetujui ke `supabase/migrations/fitur_<nama>.sql`, misalnya `fitur_c_inventory.sql`. Dengan begitu, SQL itu ikut tampil di *pull request*, dan teman yang menjadi *reviewer* bisa membacanya dengan daftar periksa yang sama.

TANDA BERHASIL

Berkas `supabase/migrations/fitur_<nama>.sql` ada di *branch*-mu dan isinya sama dengan SQL yang kamu setujui, termasuk blok `revoke` dan `grant`. Bila agen menjalankan perbaikan setelah persetujuan pertama, versi perbaikan itu yang tersimpan.

6.2 Cara Memperbarui Tipe `database.ts`

Berkas `src/types/database.ts` berisi daftar tabel, kolom, dan fungsi beserta tipenya, dibuat otomatis oleh alat `generate_typescript_types` dari Supabase MCP. Editor memakai berkas ini untuk memeriksa apakah nama fungsi dan parameter di `supabase.rpc()` benar. Setiap kali database berubah, berkas ini perlu dibuat ulang. Bila editor menandai nama fungsi atau kolom sebagai *error* setelah ada perubahan database, kirim prompt bantuan berikut dari Daftar Prompt App 4.

PROMPT MEMPERBARUI TIPE

Buat ulang src/types/database.ts lewat generate_typescript_types, lalu jalankan npx tsc --noEmit dan laporkan hasilnya.

Saat terjadi *merge conflict* di berkas ini, jangan menggabungkannya secara manual. Berkas tipe adalah hasil cetak dari database, dan database kelompok sudah berisi fungsi semua fitur. Membuatnya ulang menghasilkan versi yang lengkap dan benar.

6.3 Cara Menjalankan Advisor Keamanan

DEFINISI

Advisor di Supabase adalah kumpulan pemeriksaan otomatis yang menandai masalah keamanan dan performa di database. Security Advisor memeriksa hal seperti RLS yang mati dan fungsi yang `search_path`-nya tidak dikunci. Performance Advisor memeriksa hal seperti indeks yang hilang.

Advisor bisa dibuka di Dashboard Supabase pada menu Advisors, atau dijalankan agen lewat alat `get_advisors` dari Supabase MCP. `AGENTS.md` di starter sudah meminta agen menjalankan advisor keamanan setelah perubahan database dan melaporkan hasilnya. Bila agen belum melakukannya, kirim permintaan berikut.

CONTOH PERMINTAAN

Jalankan get_advisors untuk keamanan di proyek kelompok. Tampilkan peringatan yang menyebut fungsi, trigger, atau policy dari fitur C saja, jelaskan artinya dengan bahasa sederhana, dan usulkan perbaikan. Jangan mengubah apa pun sebelum aku setujui.

Tabel berikut memuat beberapa peringatan Security Advisor yang terkait isi modul ini. Nama lint diambil dari dokumentasi Supabase.

Peringatan	Arti dan tindakan
0011 *Function Search Path Mutable*	Ada fungsi tanpa `search_path` yang dikunci. Minta agen menambahkan `set search_path = ''` dan menulis skema lengkap.
0013 *RLS Disabled in Public*	Ada tabel di skema `public` dengan RLS mati. Seharusnya tidak muncul karena tabel tidak boleh diubah; bila muncul, tanyakan SQL mana yang mematikannya.
0024 *Permissive RLS Policy*	Ada *policy* yang terlalu longgar. Periksa apakah itu *policy* tulis `using (true)` (subbab 5.2).
0028 dan 0029 *Security Definer Function Executable* (oleh `anon` atau `authenticated`)	Fungsi `security definer` bisa dipanggil lewat API. Untuk `anon`, cabut haknya. Untuk `authenticated`, peringatan ini bisa muncul pada fungsi PRD 9.2 yang memang sengaja dipanggil aplikasi; pastikan fungsi itu memeriksa peran di awal.

Tidak semua peringatan berarti ada kesalahan, tetapi setiap peringatan harus bisa dijelaskan. Bila tidak ada yang bisa menjelaskan kenapa sebuah peringatan muncul, anggap itu masalah dan sampaikan ke kelompok.

6.4 Menguji Fungsi dengan Meniru Login Akun

Uji lewat tampilan memeriksa tombol dan formulir. Uji yang lebih penting memeriksa database secara langsung, karena orang yang berniat buruk tidak memakai tombol. Ia bisa memanggil fungsi lewat Console *browser* dengan *publishable key* yang memang terlihat publik. Di sini `execute_sql` dipakai untuk meniru keadaan itu.

DEFINISI

Meniru login berarti menjalankan SQL seolah-olah dikirim oleh akun tertentu: agen mengganti peran database menjadi `authenticated` dan mengisi identitas pengguna dengan ID akun uji, sehingga `auth.uid()` dan `private.peran_saya()` di dalam fungsi membaca akun itu. RLS, hak tabel, dan pemeriksaan di fungsi berlaku sama seperti permintaan dari aplikasi.

Anggota A mengirim prompt berikut setelah uji lewat tampilan selesai. Prompt ini dikutip dari Daftar Prompt App 4.

PROMPT UJI A: SERANGAN LEWAT DATABASE

Lewat execute_sql, tiru login Pak Joko lalu panggil ubah_status_wo untuk satu WO Tim B, dan untuk lompatan menunggu ke siap_diambil di WO Tim A. Jangan ubah data lain. Laporkan pesan error kedua uji.

Hasil yang benar: kedua panggilan ditolak dengan pesan berbahasa Indonesia. Uji pertama menguji pemeriksaan hubungan tim (subbab 3.4); bila perpindahan yang dicoba agen sah menurut PRD 7.2, pesannya "Hanya Kepala Mekanik tim ini yang boleh mengubah status WO ini". Uji kedua menguji penutup `else` (subbab 3.5) dan seharusnya menghasilkan "Perpindahan status dari menunggu ke siap_diambil tidak diizinkan".

Daftar Prompt App 4 juga memuat prompt bantuan untuk fitur C dan D. Keduanya dikutip apa adanya.

PROMPT UJI C: DATA RAHASIA DAN PERAN

Lewat execute_sql, jalankan dua uji tanpa mengubah data lain:
1. Tiru login Rina, lalu baca tabel inventory_costs. Laporkan jumlah baris yang terbaca.
2. Tiru login Pak Joko, lalu ubah kolom role miliknya menjadi owner. Laporkan pesan error-nya.

Hasil yang benar: uji 1 membaca 0 baris, uji 2 ditolak dengan pesan "Hanya Owner yang boleh mengubah peran atau status akun". Uji 2 inilah yang membuktikan `jaga_profil` berjalan tanpa `security definer` (subbab 4.4).

PROMPT UJI D: AUDIT LOG TIDAK BISA DIHAPUS

Lewat execute_sql, tiru login Rina lalu coba hapus satu baris audit_logs. Setelah itu tiru login Pak Darto dan coba hal yang sama. Jangan ubah data lain. Laporkan hasil kedua uji.

Hasil yang benar: kedua percobaan ditolak. Rina dan Pak Darto sama-sama tidak punya hak `delete` pada `audit_logs`, sehingga riwayat perubahan tidak bisa dihapus, bahkan oleh Owner.

Uji Lampu H4 dari subbab 3.6 bisa dilakukan lewat tampilan oleh anggota B: buka satu WO berstatus `dikerjakan` di dua tab dengan akun Pak Joko, lalu tambahkan Lampu H4 di kedua tab hampir bersamaan. Hasil yang benar: satu berhasil, satu ditolak dengan pesan "Stok Lampu H4 hanya 0".

PERHATIKAN

Uji ini berjalan di database kelompok yang sedang dipakai bersama. Karena itu setiap prompt uji menyertakan kalimat "jangan ubah data lain". Uji yang berhasil (tidak ditolak) mengubah data sungguhan, misalnya stok Lampu H4 menjadi 0. Sampaikan ke kelompok sebelum menjalankan uji yang mengubah data.

6.5 Bila Hasil Uji Tidak Sesuai

Bila sebuah uji yang seharusnya ditolak ternyata berhasil, berarti ada celah di SQL yang sudah berjalan di database bersama. Hentikan pekerjaan halaman, sampaikan ke kelompok, lalu minta agen menelusuri penyebabnya. Untuk *error* yang muncul di halaman, Daftar Prompt App 4 menyediakan prompt bantuan berikut.

PROMPT MENCARI PENYEBAB ERROR

Halaman ini menampilkan error berikut: [tempel pesan error]. Telusuri penyebabnya dengan membaca kode halaman, log Supabase lewat MCP, dan PRD. Jelaskan penyebabnya dengan bahasa sederhana dan usulkan perbaikannya. Jangan mengubah tabel dan jangan mengubah file milik fitur lain. Tunggu persetujuanku sebelum mengubah apa pun.

Perbaikan dari agen adalah SQL baru, sehingga ia melewati urutan yang sama sejak awal: dibaca dengan daftar periksa, disampaikan ke kelompok, disetujui, disimpan ke berkas *migration* fitur, diikuti pembaruan tipe, advisor, dan uji ulang. Daftar berikut merangkum seluruh langkah setelah SQL disetujui.

- [ ] SQL tersimpan di `supabase/migrations/fitur_<nama>.sql` dan sama dengan yang dijalankan.
- [ ] `src/types/database.ts` sudah dibuat ulang dan `npx tsc --noEmit` tidak melaporkan *error*.
- [ ] Advisor keamanan sudah dijalankan dan setiap peringatan bisa dijelaskan.
- [ ] Uji dengan meniru login akun menghasilkan penolakan yang benar, dengan pesan berbahasa Indonesia.
- [ ] Kelompok sudah diberi tahu hasilnya saat cek singkat pukul 11.30 atau 13.45.

Daftar Istilah

Istilah	Arti
`apply_migration`	Alat Supabase MCP untuk menerapkan perubahan struktur database, seperti membuat fungsi atau *trigger*.
`auth.uid()`	Fungsi Supabase yang mengembalikan ID pengguna yang sedang login.
Advisor (Security, Performance)	Pemeriksaan otomatis Supabase yang menandai masalah keamanan dan performa database.
`current_user`	Peran database yang dipakai untuk memeriksa hak saat ini; berubah menjadi pemilik fungsi di dalam fungsi `security definer`.
Database function	Sekumpulan perintah SQL bernama yang disimpan di database dan bisa dipanggil aplikasi.
`declare`	Bagian fungsi PL/pgSQL untuk menyiapkan variabel sebelum `begin`.
`execute_sql`	Alat Supabase MCP untuk menjalankan perintah SQL, dipakai juga untuk uji dengan meniru login.
`for update`	Tambahan pada `select` yang mengunci baris sampai *transaction* selesai, sehingga permintaan lain menunggu.
`generate_typescript_types`	Alat Supabase MCP untuk membuat berkas tipe TypeScript dari struktur database.
Hak `execute`	Izin menjalankan sebuah fungsi; diatur dengan `grant` dan `revoke`.
Migration	Berkas berisi perubahan struktur database yang disimpan di repo.
`new` dan `old`	Versi baru dan versi lama baris di dalam fungsi *trigger*.
Parameter	Masukan sebuah fungsi, ditulis dengan nama dan tipe, misalnya `p_jumlah int`.
Pemeriksaan hubungan	Pemeriksaan apakah pemanggil terkait dengan data tertentu, misalnya Kepala Mekanik pemilik tim WO.
`private.peran_saya()`	Fungsi bantuan yang mengembalikan peran pengguna aktif yang login, atau `null`.
`private.tim_wo()`	Fungsi bantuan yang mengembalikan `kepala_mekanik_id` sebuah WO.
PL/pgSQL	Bahasa prosedural PostgreSQL yang menambahkan variabel, percabangan, dan perulangan ke SQL.
`raise exception`	Perintah untuk menghentikan fungsi dengan pesan *error*; biasanya membatalkan seluruh *transaction*.
`returns`	Bagian kepala fungsi yang menyebut jenis hasilnya, misalnya `void`, `uuid`, atau `trigger`.
`search_path`	Daftar skema yang dicari saat nama tabel atau fungsi ditulis tanpa skema.
`security definer`	Fungsi berjalan dengan hak pemiliknya, bukan hak pemanggil.
`security invoker`	Fungsi berjalan dengan hak pemanggil; pilihan bawaan PostgreSQL.
Skema	Kelompok objek di database, misalnya `public` (terbuka lewat API) dan `private` (tidak terbuka).
`supabase.rpc()`	Perintah pustaka JavaScript Supabase untuk memanggil *database function*.
Trigger	Aturan yang menjalankan fungsi secara otomatis saat `insert`, `update`, atau `delete` terjadi pada tabel.
`using (true)`	Syarat *policy* yang selalu terpenuhi; dilarang untuk operasi tulis di aplikasi bengkel.

Daftar Pustaka

PostgreSQL Global Development Group. (t.t.). *CREATE FUNCTION* (termasuk bagian *Writing SECURITY DEFINER Functions Safely*). PostgreSQL Documentation. https://www.postgresql.org/docs/current/sql-createfunction.html

PostgreSQL Global Development Group. (t.t.). *Explicit Locking: Row-Level Locks*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/explicit-locking.html

PostgreSQL Global Development Group. (t.t.). *SELECT: The Locking Clause*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/sql-select.html

PostgreSQL Global Development Group. (t.t.). *Overview of Trigger Behavior*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/trigger-definition.html

PostgreSQL Global Development Group. (t.t.). *PL/pgSQL: Errors and Messages*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/plpgsql-errors-and-messages.html

PostgreSQL Global Development Group. (t.t.). *PL/pgSQL under the Hood: Variable Substitution*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/plpgsql-implementation.html

PostgreSQL Global Development Group. (t.t.). *System Information Functions and Operators*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/functions-info.html

Supabase. (t.t.). *Database Functions*. Supabase Docs. https://supabase.com/docs/guides/database/functions

Supabase. (t.t.). *Postgres Triggers*. Supabase Docs. https://supabase.com/docs/guides/database/postgres/triggers

Supabase. (t.t.). *JavaScript Reference: rpc()*. Supabase Docs. https://supabase.com/docs/reference/javascript/rpc

Supabase. (t.t.). *Performance and Security Advisors*. Supabase Docs. https://supabase.com/docs/guides/database/database-advisors

Supabase. (t.t.). *Model Context Protocol (MCP)*. Supabase Docs. https://supabase.com/docs/guides/getting-started/mcp